

层次方框图，描绘数据结构，组成关系用于需求分析。

层次图，表示软件结构，用于调用，适用于自顶向下设计软件过程中使用。

可移植性，表明程序转移到一个新的计算环境的可能性大小，或者表明程序可以容易的有效的在各种各样的计算环境中运行的容易程度。

可复用性，在面向对象设计中，一个类的设计应具有通用性，为开发相似的系统提供软件重用可能。

可靠性，程序在给定的时间间隔内按照规格说明书的规定成功运行的概率。随着运行时间的增加，运行时出现程序故障的概率也增加，可靠性减小。

可用性，程序在给定的时间点按照规格说明书成功运行的概率。

识别用例采用业务语言而非技术语言，采用用户观点而非系统观点，采用动宾结构而非主谓结构。

软件工程是借鉴传统工程的原则方法，以提高质量，降低成本为目的指导计算机软件开发和维护的工程学科。

软件工程的目标是达到要求的功能，取得较好的性能，开发的软件易于移植，付出较低的开发成本，需要较低的维护费用，能按时完成开发任务，及时交付使用开发的软件可靠性高。软件工程的内容开发技术和开发管理两个方面。

软件危机是指计算机开发和维护过程中所遇到的一系列严重问题，这些问题不仅是不能正常运行的软件才具有的，而是几乎所有的软件都在不同程度上存在的。

软件危机的表现

- 1.对软件开发的进度和成本估计很不准确
- 2.开发的软件产品不能满足用户的需求软件质量不可靠
- 3.软件通常没有相应文档
- 4.软件不可维护
- 5.软件的开发效率比较低

软件危机的主要原因是软件本身的特点及开发方式。

软件危机出现的原因人类智能的局限性，协同工作效率比较低。软件规模越来越大，结构越来越复

杂，软件开发管理困难，软件开发费用不断增加，软件开发技术落后，软件开发工具落后，软件开发方式落后。

题：软件生存周期：一个软件从提出开发要求开始直到该软件报废为止的整个时期。包括：可行性分析，项目开发计划，需求分析，概要设计，详细设计，编码，测试，维护。或者：可行性研究，需求分析，软件设计，编码，软件测试，软件维护

耦合是两个模块间联接程度的度量。

内容耦合：一个模块直接修改或操作另一个模块的数据

公共耦合：两个以上的模块共同引用一个全局数据项

控制耦合：一个模块向另一个模块传递一个控制信号，接受信号的模块将依据该信号值进行必要的活动

特征耦合：把整个数据结构作为参数传递，而被调用的模块只需要使用其中一部分数据元素（复杂类型的数据）

数据耦合：模块间通过参数传递基本类型的数据

结构化分析：简称 SA，面向数据流的需求分析方法。采用自顶向下逐层分解的分析策略。顶层抽象地描述整个系统，底层具体地画出系统工程地每个细节。中间层则是从抽象到具体的过渡。

使用数据流图，数据字典，作为描述工具，使用结构化语言，判定表，判定树描述加工逻辑。

软件工程是借鉴传统工程的原则和方法，以提高质量、降低成本为目的，指导计算机软件开发和维护的工程学科。（1分）

软件工程的目的是：达到要求的功能；取得较好的性能；开发的软件易于移植；付出较低开发成本；需较低的维护费用；能按时完成开发任务，及时交付使用；开发的软件可靠性高。（3分）

软件工程内容：包括开发技术和开发管理两个方面。（1分）

软件工程的三要素，过程，方法，工具。

软件工程方法论包括面向对象化方法，面向过程方法，敏捷开发方法。

面向过程方法又称为结构化分析方法，结构化分析简称 SA,面向数据流的需求分析方法，采用自顶向下逐层分解的策略，顶层橱窗的描述，整个系统底层具体画出系统工程的每个细节，中间层则从抽象到具体的过渡，使用数据流图，数据字典作为描述工具，使用结构化语言判定表判定树描述加工逻辑。

面向对象方法是将对象的属性和方法封装起来，形成信息系统的基本执行单位。再利用继承特性派生出其他执行单位，从而生成许多新的对象，将这些离散对象通过事件或消息连接起来，形成软件系统。面对方法执行过程中不由程序员控制，完全由用户交互控制。

敏捷开发来源于开发事件的总结，将整个开发过程分为一系列短的迭代过程，每个迭代过程完成一定数目的性能。迭代周期应尽量短，迭代由测试和反馈驱动，每个迭代周期产出的都是一个经过验证的可用产品，只是有可能功能不全，并且这是一个有意识持续的积极反馈的改进过程。

软件工程所面临的问题，软件费用，软件可靠性，软件可维护性，软件生产率和软件重用。

软件具有复杂性，一致性，可变性和不可见性。

软件工程的基本原则，用分阶段的生命周期计划严格管理，坚持阶段性评审，实施严格的产品控制，采用现代的程序技术，结果应能清楚地审查，开发的人少而精，承认不断改进软件工程实践的必要性。

软件的生存周期是指一个软件从提出技术开发要求开始到知道该软件报废为止的整个时期，包括可行性分析，项目开发计划，需求分析，概要设计，详细设计，编码，测试，维护。（软件定义、软件开发、软件维护）

软件生存周期阶段划分：

◇ 软件定义阶段：确定软件开发的目标及其可行性

★ 问题定义：说明软件要解决的问题

- ★ 可行性研究阶段：用最小代价在尽可能短时间内确定问题是否能解决
- ★ 需求分析：准确地回答“软件必须做什么”，即具备什么功能
- ★ 软件项目计划：从各方面对软件开发进行合理的计划
- ◇ 软件设计阶段：将需求分析时期得到的逻辑模型设计成具体计算机软件方案
 - ★ 概要设计：设计软件结构，确定功能模块及模块间的关系
 - ★ 详细设计：给出每个模块的实现算法和模块内部的数据结构等细节
 - ★ 编码
 - ★ 测试：检验是否满足用户期待，分为模块测试、组装测试、确认测试
- ◇ 运行与维护阶段：保障软件的正常运行以及对软件进行维护

软件工程是为了获得高质量软件所实施一系列活动。

软件生命周期模型 P13

- ◇ 瀑布模型：将软件生命周期的各项活动规定为依照固定顺序连接的若干阶段工作

- ★ 特点（是整体模型，开发完成后才能见到整个软件）

阶段间具有顺序性和依赖性（自上而下，相互衔接，前阶段输出是后阶段输入）

推迟实现的观点（编码前只考虑逻辑模型而不考虑具体实现）

每个阶段必须完成规定的文档，每个阶段结束前完成文档审查，及早改正错误

- ★ 局限性

阶段与阶段划分固定，阶段间产生大量的文档，增加了工作量

开发模型呈线性，开发成功尚未经过测试时，用户无法看到软件的效果

无法通过开发活动澄清本来不够确切的软件需求

顺序固定，前期造成的差错到后期损失更大，代价高，灵活性差

- ★ 优点：提供了一套工程化的管理模型，能够有效地保证软件质量，使得软件易于维护

- ◇ 增量模型：非整体开发模型，开发一部分，展示一部分 P14

1.增量构造模型：需求分析阶段和瀑布模型相似，编码和测试阶段按照增量方式开发。

在开发过程中用户能够及早发现软件中的问题

2.演化提交模型：各阶段都是增量开发的形式

3.快速原型：需求分析之前，提供用户一个最终产品的原型，迭代完善用户需求

★ **特点：**

开发初期需求定义只是用来确定软件的基本结构，用户只需要对软件需求进行大概的描述

可以按照构建的功能安排开发的优先顺序，逐个实现和交付使用

软件系统是逐渐展开，有利于降低项目的技术风险

★ **缺陷：**

要求软件的体系结构是开放的

容易退化为边做边改的模型，是软件过程的控制失去整体性

◇ **同步-稳定模型**

★ **优点：**

尽早的软件测试，而且是集成测试，降低了软件开发的風險

定期的软件集与测试，保证了交流持续性

可以及时获得软件实现的反馈新信息

◇ **螺旋模型：** P16

增加了风险分析，适用于指导大型软件项目的开发，是迭代模型

分为制定计划、风险分析、实施开发、用户评估四个阶段

螺旋线每旋转一圈表示开发出一个较前一版本更完善的新版本

◇ **喷泉模型：**

以用户需求为动力，以对象为驱动的模式

是面向对象的开发方法，开发过程具有迭代性和无间隙性

特点：

- ★ 分为分析、系统设计、软件设计和实现四个阶段，反映了并行性特点
- ★ 以分析为基础，资源消耗呈塔形，在分析阶段消耗最多
- ★ 反映了软件过程迭代的自然特性，从高层返回低层无资源消耗
- ★ 强调增量开发，分析一点设计一点
- ★ 是对象驱动的过程，对象是所有活动作用的实体，也是项目管理的基本内容
- ★ 实现可分为系统实现和对象实现，既反应全系统开发过程又反应对象族的开发和重用

◇ **基于知识的模型**：该模型建立了软件开发各阶段所需要的知识库

★ 优点

通过领域的专家系统，使得需求说明更完整、准确、无二义性

通过软件工程的专家系统，提供一个设计库支持，在开发过程中成为设计者的助手

★ 缺点

建立适合于软件设计的专家系统非常困难

建立既适合软件工程又适合应用领域的知识库非常困难

软件开发方法是克服软件危机的重要方法之一 P18

◇ 结构化方法：是面向数据流的开发方法

◇ Jackson 方法：是面向数据结构的方法

◇ 维也纳开发方法：是形式化的开发方法，基于模型

◇ 面向对象开发方法

瀑布模型是最早顺序性 依赖性。将软件生命周期划分为可行性研究与计划，需求分析，设计，编码，测试，运行维护这六个阶段。直到软件发完之后，用户才能看到整个软件。瀑布模型的特征是活动间具有顺序性和依赖性，优点是开发的各个阶段比较清晰，强调早期计划及需求调查，缺点是过度依赖于早期需求调查不适应需求的变化，单一流程不可逆，风险往往直至后期才显现出来，涉及到纠正的机会导致阻塞的状态，适合需求明确的任务。

V模型是对瀑布模型的一个变体，是一个对称结构，看起来就是一个V字，对瀑布模型的修正，强调了验证活动，非常明显的表明了测试过程中存在不同级别，并非常清晰描述了这些测试过程和开发阶段的对应关系。微模型和瀑布模型没有本质区别，V模型提供了一种将验证和确认动作应用于早期软件工程工作的直观方法。

软件开发面对的问题，需求是可变的，需求是模糊的，用户和开发者难以沟通。

增量模型，是把待开发的软件系统模块化，将每个模块看成一个增量组件，从而分批次的分析，设计，编码，测试这些增量组件。增量模型在编码和测试阶段按照增量的方式进行开发。增量模型的缺点是要求软件的体系结构是开放的，容易退化成边做边改。优点是有益于用户需求逐渐明朗，能在较短时间内向用户提交一些有用的工作产品，可以按照构建的功能安排开发的优先顺序并逐步实现和交付。开发者可以通过对诸多构建的开发逐步积累开发经验，可以使用户有比较充裕的时间学习和适应新产品。缺点是由于构件是逐渐并入已有软件系统中的，所以加入构件必须不破已构造好的系统部分，需要软件具有开放式的体系结构。在开发过程中需求变化是不可避免的。

可以采用增量模型。它是瀑布模型与原型进化模型的综合，它对软件过程的考虑是：在整体上按照瀑布模型的流程实施项目开发，以方便对项目的管理；但在软件的实际创建中，则把软件系统功能分解为许多增量构件，并以构件为单位逐个创建与交付，直到全部增量构件创建完毕，并都被集成到系统之中交付用户使用。

统一过程模型是基于建模语言的软件开发过程，提出统一建模语言UML，用例驱动，以构架为中心，采用迭代和增长的软件开发过程。优点提高了团队生产力，简洁和清晰的过程结构为开发过程提供最大的特性。缺点是缺少软件运行和支持的方面，没有支持多项目开发的结构。

同步稳定模型，要求软件及结构是开放的，克服了增量模型的缺点。

螺旋模型结合了瀑布模型和增量模型，增加了风险分析的软件过程模型项目开发分为制定计划，风险分析，实施开发，客户评估。特点是采用循环的方式逐步加深系统定义和实现的深度，同时降低风险。螺旋模型是开发大型系统和软件使用的方法，螺旋模型把原型作为降低风险的机制，保留经典生命模型中系统逐步细化的方法。

喷泉模型以用户需求为动力，以对象为驱动的模式适用于面向对象的开发方法，认为软件开发过程有两个本质特征，**迭代性和无间隙性**，将过程分为四个阶段，分析，系统设计，软件设计和实现。

基于知识的模型是把瀑布模型和专家系统结合起来，建立了各阶段所需要的知识库。

敏捷开发是为了克服传统软件过程中认识和实践的弱点而形成的，必须支持变更的思想，民营开发方法及时与变更沟通，更面临团队结构和谐的态度。

软件开发方法是一种使用早已定义好的技术及符号表示组织软件生产的过程。常用的方法有**结构化方法**，Jackson 方法**维也纳开发方法**，**面向对象的方法**。

结构化方法也是面向数据流的开发方法。Jackson 方法采用面向数据结构的方法。维也纳开发方法是一种基于模型的方法。为了实现软件工具在软件周期的各个环节自动化产生的计算机辅助软件工具 CASE

面向过程方法设计时强调模块化的思想。采用自顶向下，逐步求精的技术对系统进行划分，分解和抽象是两个基本手段。面向过程方法编程时采用单入口，单出口的控制结构，并且只包含顺序选择和循环三种结构。目标之一是使程序的控制流程线性化及程序的动态执行顺序符合静态书写结构。面向数据流方法是最具代表性的。面向过程方法的优点是以处理流程为基础简单使用，缺点是只如过程化信息，忽略信息层次关系及相互联系。

可行性分析是解决一个项目是否有可行解以及是否值得解的问题，不是解决问题，而是**确定问题是否值得去解决**。可行性分析的根本任务是对以后的行动方案提出建议。

可行性分析的步骤，1.复查系统规模和目标。2.研究正在使用的系统。3.导出新系统的高层逻辑模型。4.进一步定义问题。5.导出和评价供选择的解法。6.推荐行动方针，7.草拟开发计划。8.书写计划任务书。9.提交审查。

可行性分析技术可行，经济可行，运行可行，法律可行，开发方法可行。**技术可行性是最难决断和最关键的问题。**

数据流图是描述数据在系统中流动和处理的情况，**描述了对数据进行加工处理的过程**。数据字典是定义系统中使用的数据库。数据字典对数据流图中包含的所有元素的定义的集合是关于数据库的信息的

集合。

系统流程图是概括的描述物理系统的传统工具，反映了系统的主体框架，基本思想是用图形符号以黑盒子形式描述系统的各个部件，表示数据在各个部之间之间的流动情况。

在数据流图中没有任何具体的物理部件，**数据流图与数据字典一起构成了系统的逻辑模型。**

数据字典的内容，数据元素，数据流，数据存储，处理。

需求分析是软件定义时期的最后一个阶段，它的基本任务不是确定系统怎样完成它的工作，而是确定系统必须完成哪些工作，**准确的回答系统必须做什么的问题。**

软件的需求说明书应该包括数据描述，功能描述和性能描述。

需求分析的过程，获取用户的需求，分析用户需求，编写需求文档，评审需求文档，需求管理。

需求分析应该建立的三种模型，数据模型，功能模型和行为模型，数据模型使用实体联系图，功能模型使用数据流图，行为模型使用状态转换图。

层次方框图是组成不是调用，用于需求分析阶段。

结构化方法等于结构化分析+结构化设计+结构化实现。

验证软件需求的正确性

- ✧ 一致性：所有需求必须是一致的，任何一条需求不能和其他需求互相矛盾
- ✧ 完整性：需求必须是完整的，规格说明书应该包括用户需要的每一个功能或性能
- ✧ 现实性：制定的需求应该能用现有的硬件和软件技术实现
- ✧ 有效性：必须证明需求是正确有效的，确实能解决用户面对的问题

概要设计是对软件系统的总体设计建立目标系统的**逻辑模型**。设计实现模块的算法和数据结构。

启发规则

- ★ 改进软件结构提高模块独立性
- ★ 模块规模应该适中
- ★ 深度、宽度、扇入和扇出应适当

深度：表示软件结构中控制的层数

宽度：软件结构内同一个层次上的模块总数的最大值

扇出：一个模块直接控制(调用)的模块数目，扇出过大意味着块过分复杂，一般一个设计得好的典型系统的平均扇出是 3 或 4，扇出上限是 5 到 9

扇入：指有多少上级模块调用它，扇入大说明共享该模块的上级模块数目多。好的软件结构顶层扇出比较高，中层扇出比较少，底层扇入到公共的实用模块中，即底层模块有高扇入。

★ 模块的作用域应该在控制域范围内

作用域：指受该模块内一个判定影响的所有模块的集合

控制域：是这个模块本身以及所有直接或间接从属于它的模块的集合

★ 力争降低模块接口的复杂程度

★ 设计单入口单出口的模块

★ 模块功能应该可以预测

描绘软件结构的图形工具：

✧ 层次图：用方框和连线表示上下层的调用关系

✧ HIPO 图：层次图加编号，具有可追踪性

✧ 结构图

不仅描述调用关系，还描述传递的信息和调用方式

箭头代表调用过程传递的信息，尾部空心代表数据，实心代表控制信息

模块调用分类：简单调用、循环调用、选择调用

✧ 面向数据流的设计方法

★ 数据流图分类

变换流：由输入、变换中心和输出三部分组成(外部表示 内部表示 外部表示)

事务流：在多种事务中选择一个执行，包括输入、事务处理中心、多个处理路径、输出

变换分析：把具有变换流特点的数据流图映射成模块结构图(软件结构)

概要设计的原则，模块化，抽象，逐步求精，信息隐藏和局部化，模块独立，启发原则。

详细设计的目的：确定怎样具体地实现所要求的系统，得出对目标的精确描述

详细设计的任务

- ✧ 过程设计：即设计软件体系结构中所包含的每个模块的实现算法
- ✧ 数据设计：设计软件数据结构
- ✧ 接口设计：设计软件内部各模块之间的接口

详细设计的工具

- ✧ 程序流程图

三种基本控制结构（顺序、选择、循环）两种扩展控制结构（Do-case 多分支、Do-until 循环）

椭圆代表开始和结束、矩形方框代表处理、菱形代表判断

耦合是两个模块间连接程度的度量，如果两个模块间关联度越大，联系越紧密，耦合度也越大。

耦合包括内容耦合，公共耦合，控制耦合，特征耦合和数据耦合。耦合性逐渐变弱，独立性逐渐增强。

内容耦合一个模块直接修改或操作另一个模块的数据。

公共耦合两个以上的模块共同引用一个全局数据项。

控制耦合，一个模块向另一个模块传递一个控制信号，接受信号的模块将依据该信号值进行必要的活动。

特征耦合把整个数据结构作为参数传递而被调用的模块只需要使用其中的一部分数据元素。

数据耦合模块间通过参数传递基本类型的数据。

尽量使用数据耦合，少用控制耦合，限制公共耦合的范围，避免使用内容耦合。

内聚是一个模块内部各元素之间彼此结合的紧密程度。

偶然内聚，逻辑内聚，时间内聚，过程内聚，通信内聚 顺序内聚和功能内聚。内聚性逐渐增高，独立性逐渐增强。

偶然内聚是模块中的各成分之间没有任何联系。

逻辑内聚几个逻辑上相关的功能被放在同一模块中，但这些功能并无直接关系。

时间内聚，模块完成的功能必须在同一时间内执行。

过程内聚，构建或操作的组合方式是允许在调用前面的构建或操作之后，马上调用后面的构建或操作，即使两者之间没有数据进行传递。

通信内聚，模块的所有成分都在同一数据集或生成同一数据集。

顺序内聚，模块的各个省份和统一功能密切相关，一个成分的输出作为另一个成分的输入。

功能内聚，模块的所有成分对于完成单一的功能都是必须的，缺一不可，是最理想的内聚类型。

详细设计是设计每一个模块的实现算法及过程的详细描述。工具图形工具有程序流程图，结构流程图，问题分析图，列表工具有判定表，判定树，语言工具过程定义语言。

软件测试的目的是在软件投入生产性运行之前尽可能多的发现软件中的错误，是保证软件质量的关键步骤，是对软件规格说明设计编码的最后复审。

按质量因素可以分为性能测试，功能测试，可靠性测试，可用性测试，安全性测试。

软件测试是为了发现程序中的错误而执行程序的过程不能证明程序中没有错误。

静态测试以人工测试为主。走查的对象只是代码不包括设计文档。代码走查以小组会议的形式进行。

动态测试需要通过实际运行被测程序来发现问题。

动态测试分为**黑盒测试**和**白盒测试**。**黑盒测试**是测试人员把测试的系统看成一个黑盒子，不关心盒子内部的结构和内部特性，只是关注产品的输入输出数据，从而检查产品是否符合它的功能说明。**白盒测试**是测试人员把测试的系统看成一个透明盒子，关心盒子的内部细节和逻辑结构。

黑盒测试又称为功能测试或数据驱使测试，用于测试过程的后期。

用于已知产品应具有的功能，用于发现功能不正确或遗漏了功能界面错误，数据结构错误或外部数据库访问错误，性能错误，初始化和终止错误。

黑盒测试的基本流程：设计一组测试用例用例包括输入数据和预期输出，用各个测试用例的输入数据实际运行被测程序检测实际输出结果与预测结果是否一致，不一致则认为有错。黑盒测试测试用例的设计方法等价划分，边界值分析和错误推测。等价划分最常用。

等价划分分为有效等价类和无效等价类。有效等价类是一个，无效等价类是两个。边界值分析确定边界情况，测试数据刚好等于边界值小于边界值，刚刚大于边界值。

黑盒测试的优点是比较简单，无需了解程序内部的代码及实现与软件内部实现无关，从用户角度出发，缺点是不可能覆盖所有代码，自动化测试的复用性较低。

白盒测试又称为结构测试，主要在测试过程的早期阶段进行，把程序看成装在一个透明的白盒子里，测试者完全知道程序的结构和处理算法，按照程序内部逻辑测试程序，检测程序中主要执行通路是否都能按照预定的要求正确工作。

逻辑覆盖包括语句覆盖，判定覆盖，条件覆盖，判定/条件覆盖，条件组合覆盖，路径覆盖。发现错误的能力逐渐增强。控制结构测试包括基本路径测试，条件测试，循环测试。

语句覆盖（点覆盖），设计若干个测试用例，运行被测程序，使每一个可执行语句至少执行一次。

判定覆盖（边覆盖），设计的测试用例保证程序中的每个判断的至少执行一次。

条件覆盖，使得程序中每个判断条件的所有可能结果至少取到一次，又使每次判断的每个分支至少通过一次。

判定/条件覆盖：设计的测试用例，保证程序的每个判断和条件都能尽可能多的执行。

条件组合覆盖每个判定中各条件的每一种组合至少出现一次。

路径覆盖使程序中每一条可能的路径至少执行一次。

基本路径测试的流程，一，根据过程设计结果画出相应的流图。二，计算流图的环形复杂度。三，确定独立路径的基本集合。四，设计可强制执行基本集合中每条路径的测试用例。

软件测试的步骤，模块测试，集成测试，确认测试，阿尔法测试和 beta 测试，平行运行，回归测试。

软件测试步骤

◇ 模块测试

◇ 集成测试

★ 非渐增式测试方法：先分别测试再集成

★ 渐增式测试方法：测试一个加入下一个

自顶向下集成：深度优先策略和广度优先策略

自底向上集成

◇ 确认测试

★ 验证：为了保证软件正确实现了某个特定要求而进行的一系列活动

★ 确认：为了保证软件确实满足了用户需求而进行的一些列活动

◇ Alpha 和 Beta 测试

★ Alpha：开发者在场的情况下用户在指导下进行测试

★ Beta：开发者不在场

◇ 平行运行

回归测试

模块测试（单元测试）的目的是保证每一个模块作为一个单元的正常运行，在模块测试中发现的往往是编码和详细设计中的错误。

集成测试的目的是发现与接口有关的问题，测试方法非渐增式测试方法，渐增式测试方法在集成测试中发现的往往是概要设计的错误。

确认测试，目的验证系统确实能够满足用户的需要，通常使用黑盒测试法发现的是需求分析的错误。

面向对象方法的四个要点，一，认为客观世界是由各种对象组成的，任何事物都是对象，复杂的对象可以有比较简单的对象，以某种方式组合。二把所有的对象都划分成各种对象类，每个类都定义了一组数据和一组方法。三，按照子类与父类的关系把若干对象类组成一个层次结构的系统。四，对象彼此之间只能通过传递消息互相联系。

面向对象就是既使用对象又使用类和继承等机制，而且对象之间仅能通过传递消息实现彼此通信。

面向对象的优点，稳定性好，可重用性好，交易开发大型软件，可维护性好。

对象等于对象名+数据+操作。

用例图主要用于需求分析阶段，从用户的角度出发描述系统的功能。

类图用来描述类与类之间的静态关系。

面向对象分析的三个模型，对象模型，动态模型和功能模型，对象模型描述系统数据结构，动态模型描述系统控制结构，功能模型描述系统功能。

面向对象分析的五个层次，主题层类与对象层，结构层，属性层和服务层。

类：具有相同属性和服务的一组对象的集合，它为属于该类的全部对象提供了统一的抽象描述

属性：属性是对客观世界实体所具有的性质的抽象，是类中所定义的数据

方法：对象所能执行的操作称为方法，也就是类中所定义的服务。方法描述了对对象执行的功能、响应消息的操作

消息：对象之间通信的手段，是一个对象要求另一对象执行类中定义的某个操作的规格说明

- ◇ 接受消息的对象
- ◇ 消息标识符(即消息名)
- ◇ 零个或多个变元

继承：直接获得已有的性质和特征，而不必重复它们。

父类：一个类的上层是父类

子类：一个类的下层是子类

超类：父类的父类称为超类

单继承：一个类只允许有一个父类，即类等级为树形结构

多继承：一个类允许有多个父类

继承的优点

- ★ 使得相似的对象可以共享程序代码和程序结构，大大减少程序中的冗余信息
- ★ 便于软件修改维护
- ★ 用户在开发新系统时不必从零开始，可以继承原有相似功能或者从类库中
- ★ 选取需要的类，再派生新类

服务：某个对象所具有的特定的行为

封装：封装是把对象的属性和实现操作的代码结合成一个独立的系统单位，并尽可能隐藏对象的内部细节，对象以外的部分不能直接作用于对象的内部数据(属性)，对象间的通信只能通过明确的消息来进行。

多态性：指对象根据所接受的消息而做出动作，同样的消息为不同的对象接受时可导致完全不同的行动

重载

★ 函数重载：在同一作用域，若干个参数特征不同的函数可以使用相同的函数名

★ 运算符重载：同一个运算符可以施加于不同类型的操作数上面。

UML 的表示法由 UML 图、视图、模型元素、通用机制、扩展机制构成

UML 模型的五类图：用例图、静态图（类图、对象图、包图）、行为图（状态图、活动图、顺序图、协作图）、交互图、实现图